

UNIVERSIDAD DE SANTIAGO DE CHILE
FACULTAD DE INGENIERÍA
Departamento de Ingeniería Informática



Proyecto de investigación
Maximum Clique

Integrantes: Ricardo Riveros
Gonzalo Ahumada
Daniel Muñoz
Asignatura: Optimización en Ingeniería
Profesora: Mónica Villanueva
Ayudante: Sebastián Aliaga

9 de junio de 2026

Tabla de contenidos

Resumen	1
1. Introducción	2
2. Estado del Arte	4
2.1. Formulación cuadrática continua: Teorema de Motzkin-Straus	5
2.2. Contextos de aplicación	6
2.2.1. Teoría de códigos y diseño de códigos correctores de errores . . .	6
2.2.2. Bioinformática: alineamiento de estructuras moleculares	6
3. Método	8
3.1. Formulación Matemática del Problema	8
3.1.1. Formulación como Programa Lineal Entero (ILP)	8
3.1.2. Relajación Lineal Continua (LP)	9
3.2. Procedimiento de Resolución	9
3.3. Dificultades experimentales	10
3.4. Diagrama de Flujo del Proceso	11
4. Resultados y Análisis	14
4.1. Descripción de la instancia	14
4.2. Dimensiones del modelo ILP	14
4.3. Resultados de la resolución exacta: comparativa de solvers	14
4.4. Análisis de la relajación LP	15
4.5. Visualización	16
5. Discusión	17
5.1. Efectividad de la formulación ILP	17
5.2. Rol del solver HiGHS	17
5.3. Ventajas del enfoque con Pyomo	17
5.4. Comparación con enfoques alternativos	18
5.5. Limitaciones	18

6. Conclusiones	19
6.1. Trabajo futuro	20
7. Apéndice	21
7.1. Entorno de ejecución	21
7.2. Fragmento de código: modelo ILP en Pyomo	21
7.3. Instancia keller4: formato DIMACS	22
Bibliografía	23

Resumen

Este trabajo aborda el Problema del Clique Máximo (MCP), un problema clásico de optimización combinatoria clasificado como NP-duro, mediante una formulación de Programación Lineal Entera (ILP) resuelta con herramientas de modelado algebraico. Se implementa el modelo matemático en Python utilizando el framework Pyomo (Bynum et al., 2021) y el solver de alto rendimiento HiGHS (Huangfu & Hall, 2018), que internamente aplica el método de Ramificación y Acotamiento (Branch & Bound) con relajación LP, planos de corte y heurísticas. La metodología se evalúa sobre la instancia *keller4* del benchmark DIMACS (Mascia, 2014), un grafo de 171 vértices y 9 435 aristas basado en la conjetura de Keller sobre teselaciones con hipercubos. Los resultados experimentales confirman la obtención de la solución óptima $\omega = 11$ en un tiempo de entre 12 y 18 segundos según la ejecución, mientras que el solver GLPK, ejecutando idéntica formulación, encuentra un clique factible del mismo tamaño pero no logra probar optimalidad dentro de un límite de 5 minutos. El análisis de la relajación LP revela una brecha de integralidad del 87.1 %, característica de la debilidad intrínseca de la formulación LP estándar para el MCP. Se concluye que la combinación de modelado algebraico declarativo con solvers MIP modernos constituye un enfoque efectivo y reproducible para la resolución exacta de problemas de optimización combinatoria.

Palabras clave: Clique Máximo, Programación Lineal Entera, Branch & Bound, Modelado Algebraico, Optimización Combinatoria, Complejidad Computacional, Benchmark DIMACS.

1. Introducción

El análisis de estructuras complejas mediante la teoría de grafos ha cobrado gran relevancia en la última década debido al auge de la ciencia de datos y el análisis de redes. Dentro de este ámbito, el Problema del Clique Máximo (Maximum Clique Problem, MCP) se define formalmente como la búsqueda del subgrafo completo de mayor cardinalidad dentro de un grafo no dirigido $G = (V, E)$, donde todos los vértices del subgrafo están mutuamente conectados (Pardalos & Xue, 1994). Más precisamente, un *clique* en G es un subconjunto $C \subseteq V$ tal que $\{u, v\} \in E$ para todo par $u, v \in C$, y el MCP consiste en encontrar el clique C^* de máxima cardinalidad $\omega(G) = |C^*|$.

La relevancia práctica del problema se sustenta en su amplia aplicabilidad en áreas como la bioinformática, la teoría de códigos, el reconocimiento de patrones, el diagnóstico de fallos, la visión por computadora y la economía (Pardalos & Xue, 1994; Singh & Govinda, 2014). Sin embargo, el MCP pertenece a la familia de problemas NP-duros (Bomze et al., 1999; Pardalos & Xue, 1994), lo que implica que el tiempo computacional crece exponencialmente con el tamaño del grafo y que no se conoce un algoritmo que lo resuelva en tiempo polinomial para instancias arbitrarias. Esta intratabilidad justifica el estudio de estrategias eficientes de poda, ramificación y relajación.

Objetivo general. Formular el MCP como un Programa Lineal Entero (ILP) y resolverlo de manera exacta sobre una instancia representativa del benchmark DIMACS, evaluando además la calidad de la formulación a través de su relajación lineal continua.

Objetivos específicos.

- Implementar el modelo ILP del MCP en Python utilizando el framework de modelado algebraico Pyomo.
- Resolver la instancia *keller4* del benchmark DIMACS mediante el solver de alto rendimiento HiGHS, que aplica internamente Ramificación y Acotamiento (Branch & Bound) con relajación LP, planos de corte y heurísticas.

- Calcular y analizar la brecha de integralidad entre la solución entera y la relajación continua, contrastando los resultados con el óptimo conocido reportado en la literatura.

Pregunta de investigación. ¿Es suficiente la formulación ILP estándar del MCP, resuelta mediante un solver MIP moderno, para alcanzar la solución óptima conocida de una instancia densa del benchmark DIMACS en tiempos razonables, y qué calidad ofrece su relajación lineal continua como cota superior?

El resto del documento se organiza como sigue: la Sección 2 presenta el estado del arte y dos contextos de aplicación; la Sección 3 describe el método y la formulación matemática; finalmente, se presentan los resultados experimentales, la discusión y las conclusiones.

2. Estado del Arte

La revisión de la literatura en torno al Problema del Clique Máximo (MCP) revela un esfuerzo de más de seis décadas por encontrar soluciones algorítmicas eficientes a este desafío NP-duro (Bomze et al., 1999; Pardalos & Xue, 1994). A lo largo de los años, los enfoques han evolucionado desde la combinatoria discreta pura hacia técnicas de optimización continua y heurísticas avanzadas. A continuación, se detallan los avances predominantes:

1. **Complejidad y Modelos Matemáticos:** El MCP es computacionalmente equivalente al problema del Conjunto Independiente Máximo (Maximum Independent Set) y al problema de Cobertura de Vértices Mínima (Minimum Vertex Cover) (Pardalos & Xue, 1994). El problema se modela comúnmente mediante programación entera (como un problema binario discreto). Una formulación alternativa de naturaleza continua, basada en el teorema de Motzkin-Straus, se detalla en la Sección 2.1. Desde la perspectiva de la complejidad, no solo es NP-duro encontrar la solución exacta, sino que se ha demostrado que es imposible aproximar el tamaño del clique máximo dentro de un factor constante en tiempo polinomial, a menos que $P = NP$ (Pardalos & Xue, 1994).
2. **Métodos de Solución Exactos:** En el ámbito de la optimización exacta, el enfoque principal ha sido el método de Ramificación y Acotamiento (*Branch & Bound*). Históricamente, el algoritmo de Carraghan y Pardalos (1990) sentó las bases algorítmicas para los solucionadores modernos, utilizando enumeración parcial y acotamiento agresivo (Szabó & Zaválnij, 2018). Solucionadores actuales de vanguardia como *cliquer* (Östergård), MCR (Tomita) y BBMC (San Segundo) son descendientes y versiones refinadas de este enfoque clásico (Szabó & Zaválnij, 2018). Recientemente, se ha demostrado que reformular el problema de optimización en una serie de problemas de decisión parametrizados (búsqueda de un clique de tamaño específico k) simplifica la estructura del problema y permite reducir drásticamente el espacio de búsqueda (Szabó & Zaválnij, 2018).

3. **Enfoques Heurísticos:** Dada la dificultad de los métodos exactos en grafos muy grandes, la literatura ha propuesto múltiples heurísticas para aproximar soluciones. Las más comunes son las heurísticas codiciosas secuenciales (*sequential greedy heuristics*), que construyen cliques añadiendo el mejor vértice disponible ("Best in") o eliminando el peor ("Worst out") (Pardalos & Xue, 1994). Para mejorar estas aproximaciones, se aplican algoritmos de búsqueda local (como intercambios- k), búsquedas aleatorias (*Randomized algorithms*), búsqueda Tabú y redes neuronales (Pardalos & Xue, 1994). Enfoques más recientes aprovechan el cálculo de Conjuntos Independientes Mínimos (Minimal Independent Sets) en cada paso recursivo, descartando tempranamente nodos que no pueden mejorar la mejor solución actual (poda), reduciendo eficientemente el árbol de exploración (Singh & Govinda, 2014).
4. **Casos Polinomiales y Grafos Especiales:** Aunque el problema es NP-duro en grafos arbitrarios, el estado del arte reconoce que puede ser resuelto en tiempo polinomial en clases de grafos con topologías especiales. Destacan de manera fundamental los grafos perfectos (*perfect graphs*), grafos bipartitos, grafos de intervalos y grafos triangulados (Pardalos & Xue, 1994). En estas estructuras, la relajación lineal del problema entero entrega soluciones óptimas con valores enteros de manera natural (Pardalos & Xue, 1994).

2.1. Formulación cuadrática continua: Teorema de Motzkin-Straus

Una formulación alternativa del MCP, ampliamente referenciada en la literatura, proviene del teorema de Motzkin-Straus (Motzkin & Straus, 1965), que establece una equivalencia exacta entre el problema combinatorio discreto y un problema de programación cuadrática continua. Sea A la matriz de adyacencia del grafo G y Δ el simplex estándar $\Delta = \{x \in \mathbb{R}^n : \sum_{i=1}^n x_i = 1, x_i \geq 0\}$. El teorema establece que:

$$\frac{1}{2} \left(1 - \frac{1}{\omega(G)} \right) = \max_{x \in \Delta} \frac{1}{2} x^T A x, \quad (1)$$

donde $\omega(G)$ denota el tamaño del clique máximo de G . Esta reformulación transforma el problema discreto en uno de optimización continua no lineal, lo cual permite emplear técnicas de programación cuadrática para su resolución (Bomze et al., 1999; Pardalos & Xue, 1994). Aunque esta formulación no se implementa en el presente trabajo, su existencia evidencia que el MCP admite paradigmas de modelado más allá de la programación lineal entera, lo cual ha motivado numerosas extensiones, como relajaciones semidefinidas y variantes regularizadas del problema cuadrático (Bomze et al., 1999).

2.2. Contextos de aplicación

A continuación se desarrollan dos contextos en los que el MCP aparece como modelo central, justificando su interés práctico más allá del valor teórico.

2.2.1. Teoría de códigos y diseño de códigos correctores de errores

El MCP surge de manera natural en la construcción de códigos binarios de longitud n y distancia mínima de Hamming d . Se define el grafo $G_{n,d} = (V, E)$ con $V = \{0, 1\}^n$ y $\{u, v\} \in E$ si y solo si $d_H(u, v) \geq d$. Un clique en $G_{n,d}$ corresponde a un conjunto de palabras código mutuamente separadas a distancia $\geq d$; el clique máximo determina el tamaño $A(n, d)$ del código óptimo, cantidad de interés fundamental en teoría de la información (Bomze et al., 1999).

Este vínculo es particularmente relevante para la familia de instancias *Keller* utilizada en este trabajo. Los grafos G_n de Keller modelan la conjetura sobre teselaciones del espacio $\mathbb{R}^n$ por hipercubos unitarios trasladados: la existencia de un clique de tamaño 2^n en G_n implica un contraejemplo a la conjetura para dimensión n (Lagarias & Shor, 1992). Resolver el MCP en estas instancias ha sido decisivo para refutar la conjetura en dimensiones $n \geq 8$ y para estudiar empaquetamientos densos relacionados con códigos.

2.2.2. Bioinformática: alineamiento de estructuras moleculares

En biología computacional, el MCP se utiliza para encontrar la *subestructura común máxima* (Maximum Common Substructure, MCS) entre dos moléculas o

proteínas. Dadas las estructuras $\mathcal{A}$ y $\mathcal{B}$, se construye un *grafo de productos* G_P cuyos vértices son pares (a, b) con $a \in \mathcal{A}$, $b \in \mathcal{B}$ compatibles (mismo tipo de átomo o residuo), y dos vértices (a_1, b_1) y (a_2, b_2) se conectan si las distancias internas $\|a_1 - a_2\|$ y $\|b_1 - b_2\|$ coinciden dentro de una tolerancia. Un clique máximo en G_P representa el subconjunto máximo de pares mutuamente consistentes, es decir, la mejor superposición posible (Bomze et al., 1999; Gardiner et al., 1997).

Esta formulación es la base de algoritmos de *docking* molecular y descubrimiento de fármacos, donde identificar farmacóforos comunes entre ligandos activos guía el diseño de nuevas moléculas terapéuticas (Gardiner et al., 1997).

3. Método

Para la realización de este estudio, la metodología se ha estructurado de la siguiente manera:

3.1. Formulación Matemática del Problema

Sea $G = (V, E)$ un grafo no dirigido con conjunto de vértices $V = \{1, 2, \dots, n\}$ y conjunto de aristas E . Se define $\bar{E} = \{(i, j) : i, j \in V, i < j, (i, j) \notin E\}$ como el conjunto de pares no adyacentes (cada par considerado una sola vez), es decir, los pares de vértices que **no** están conectados en G .

3.1.1. Formulación como Programa Lineal Entero (ILP)

El MCP se formula como un problema de Programación Lineal Entera Binaria (Binary Integer Linear Program, BILP). Se introduce una variable de decisión binaria x_i para cada vértice $i \in V$, donde $x_i = 1$ si el vértice i pertenece al clique, y $x_i = 0$ en caso contrario (Nemhauser & Wolsey, 1988; Pardalos & Xue, 1994).

Función objetivo:

$$\text{máx} \quad z = \sum_{i \in V} x_i \quad (2)$$

Restricciones de adyacencia:

$$x_i + x_j \leq 1, \quad \forall (i, j) \in \bar{E} \quad (3)$$

Restricciones de integralidad:

$$x_i \in \{0, 1\}, \quad \forall i \in V \quad (4)$$

La función objetivo (2) maximiza el número de vértices seleccionados. Las restricciones (3) garantizan que si dos vértices i y j **no** son adyacentes (es decir, $(i, j) \in \bar{E}$), entonces ambos no pueden pertenecer simultáneamente al clique. Las restricciones (4) aseguran que las variables sean binarias. Toda solución factible de este modelo corresponde a un clique en G , y la solución óptima es el clique de cardinalidad máxima (Bomze et al., 1999; Pardalos & Xue, 1994).

3.1.2. Relajación Lineal Continua (LP)

Para obtener cotas superiores del valor óptimo entero, se resuelve la relajación lineal del problema (2)–(4), reemplazando la restricción de integralidad por su envolvente continua (Nemhauser & Wolsey, 1988; Schrijver, 1986):

$$0 \leq x_i \leq 1, \quad \forall i \in V \quad (5)$$

Sea z_{LP}^* el valor óptimo de la relajación y z_{ILP}^* el del problema entero original. Dado que la región factible del LP contiene a la del ILP, se cumple la relación fundamental:

$$z_{ILP}^* \leq z_{LP}^* \quad (6)$$

Esta propiedad es la base teórica del método de Ramificación y Acotamiento: si la cota superior z_{LP}^* de un subproblema no supera la mejor solución entera encontrada hasta el momento, dicho subproblema puede descartarse sin pérdida de optimalidad (*poda por cota*) (Nemhauser & Wolsey, 1988; Wolsey, 1998).

3.2. Procedimiento de Resolución

1. **Herramientas de desarrollo:** Los experimentos se implementaron en Python 3.12, utilizando Pyomo (Bynum et al., 2021) como framework de modelado algebraico y HiGHS (Huangfu & Hall, 2018) como solver MIP. Python ofrece una robusta capacidad para la formulación rápida de modelos de optimización y el manejo eficiente de matrices de adyacencia.
2. **Instancia de prueba:** Se utilizó la instancia *keller4* del 2nd DIMACS Implementation Challenge (Johnson & Trick, 1993; Mascia, 2014), un grafo de 171 vértices y 9 435 aristas ampliamente reconocido en la literatura para evaluar la búsqueda exacta de cliques (Pardalos & Rodgers, 1992).
3. **Procedimiento algorítmico central:** La resolución se basó en el método exacto de Ramificación y Acotamiento (*Branch & Bound*) (Carraghan & Pardalos, 1990;

Pardalos & Rodgers, 1992), ejecutado internamente por HiGHS con planos de corte (Gomory, MIR) y heurísticas de factibilidad.

4. **Cálculo de la relajación:** En cada nodo del árbol B&B, el solver resuelve la relajación lineal continua del subproblema entero (Nemhauser & Wolsey, 1988), obteniendo una cota superior z_{LP}^* del óptimo entero del subproblema.
5. **Proceso de ramificación:** Cuando la solución de la relajación contiene una variable fraccionaria, el solver selecciona una variable x_k con valor no entero y crea dos subproblemas mutuamente excluyentes ($x_k = 0$ y $x_k = 1$) (Carraghan & Pardalos, 1990; Nemhauser & Wolsey, 1988). Los subproblemas se podan por infactibilidad, por cota (cuando z_{LP}^* no mejora la incumbente) o por encontrar una solución entera factible.

3.3. Dificultades experimentales

Durante la implementación y evaluación del modelo se enfrentaron tres dificultades principales, junto con la forma en que se abordaron:

1. **Debilidad de la relajación LP.** La formulación ILP estándar (Ecuaciones (2)–(4)) genera una relajación lineal cuya cota es muy floja respecto al óptimo entero (brecha del 87,1 % en *keller4*, ver Tabla 3). Esto se abordó delegando la exploración del árbol Branch & Bound al solver HiGHS, que combina planos de corte (Gomory, MIR) y heurísticas de factibilidad para cerrar la brecha mucho antes que un esquema B&B puro.
2. **Falta de convergencia de GLPK.** En las pruebas iniciales, GLPK no logró probar optimalidad sobre *keller4* dentro de un límite de 300s, terminando con estado *feasible*. La dificultad se manejó contrastando el resultado de HiGHS contra el óptimo conocido del benchmark DIMACS ($\omega = 11$) para certificar la validez de la respuesta, y reservando GLPK únicamente como punto de comparación de desempeño entre solvers.

3. **Validación post-hoc del clique.** Para descartar errores en la lectura de la solución entera, se implementó una verificación del subgrafo inducido por los vértices seleccionados, comprobando que contiene exactamente $\binom{\omega}{2}$ aristas, todas presentes en el grafo original G .

Algorithm 1 Ramificación y Acotamiento (Branch & Bound) para el MCP

Require: Grafo $G = (V, E)$, formulación ILP del MCP

Ensure: Clique máximo x^* con cardinalidad $\tilde{z}$

```

1:  $L \leftarrow \{P_0\}$   $\triangleright$  Lista de subproblemas activos
2:  $\tilde{z} \leftarrow 0$   $\triangleright$  Mejor solución entera
3:  $x^* \leftarrow \emptyset$ 
4: while  $L \neq \emptyset$  do
5:   Seleccionar y remover un subproblema  $P$  de  $L$ 
6:   Resolver relajación LP de  $P$ : obtener  $x', z'$ 
7:   if  $P$  es infactible then
8:     descartar  $P$   $\triangleright$  Poda por infactibilidad
9:   else if  $z' \leq \tilde{z}$  then
10:    descartar  $P$   $\triangleright$  Poda por cota
11:   else if  $x'$  es entera then
12:      $\tilde{z} \leftarrow z', x^* \leftarrow x'$   $\triangleright$  Actualizar incumbente
13:   else
14:     Elegir variable fraccionaria  $x_k$  de  $x'$ 
15:      $L \leftarrow L \cup \{P|_{x_k=0}, P|_{x_k=1}\}$   $\triangleright$  Ramificar
16:   end if
17: end while
18: return  $\tilde{z}, x^*$ 

```

3.4. Diagrama de Flujo del Proceso

La Figura 1 presenta el diagrama de flujo del procedimiento implementado en el notebook experimental. Se utiliza Pyomo como framework de modelado algebraico

y HiGHS como solver MIP, el cual resuelve internamente mediante Branch & Bound con relajación LP, cortes y heurísticas.

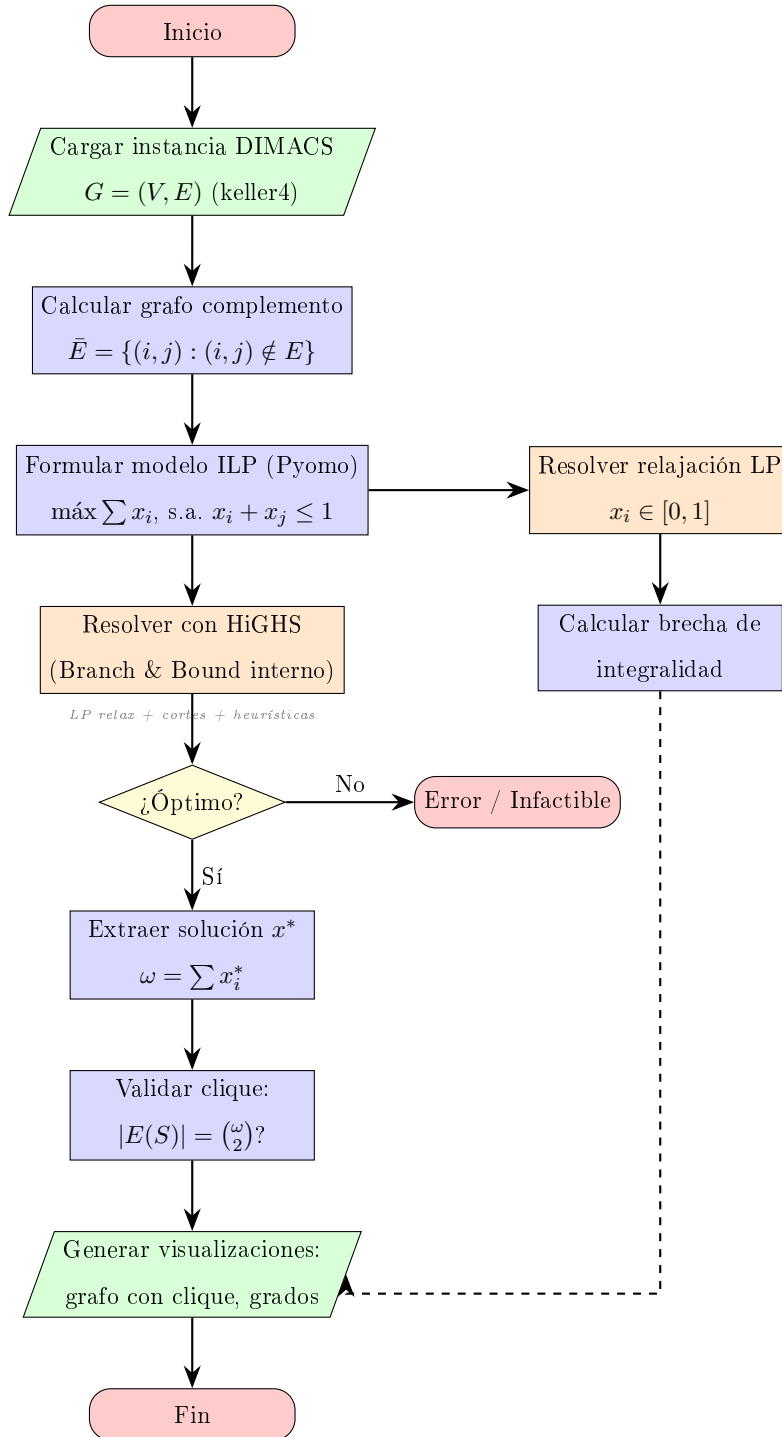


Figura 1: Diagrama de flujo de la implementación con Pyomo y HiGHS para el MCP.

4. Resultados y Análisis

En esta sección se presentan los resultados obtenidos al resolver el Problema del Clique Máximo sobre la instancia *keller4* del benchmark DIMACS (Mascia, 2014), utilizando la formulación ILP descrita en la Sección 3.1 implementada en Pyomo (Bynum et al., 2021) con el solver HiGHS (Huangfu & Hall, 2018).

4.1. Descripción de la instancia

La instancia *keller4* pertenece a la familia Keller, basada en la conjetura de Keller sobre teselaciones del espacio mediante hipercubos (Mascia, 2014). Sus características se resumen en la Tabla 1.

Tabla 1: Características de la instancia *keller4*.

Propiedad	Valor
Vértices ($ V $)	171
Aristas ($ E $)	9 435
Densidad	0.6491
Clique máximo conocido (ω)	11
Aristas del complemento ($ \bar{E} $)	5 100

4.2. Dimensiones del modelo ILP

El modelo formulado en Pyomo contiene 171 variables binarias de decisión (una por vértice) y 5 100 restricciones de no-adyacencia, correspondientes a las aristas del grafo complemento $\bar{G}$. La razón de restricciones por variable es de 29,8, lo que indica un modelo fuertemente restringido.

4.3. Resultados de la resolución exacta: comparativa de solvers

Para evaluar el efecto del solver MIP en el tiempo de resolución, se ejecutó el mismo modelo ILP (*Pyomo, idéntica formulación*) con dos solvers: HiGHS (Huangfu &

Hall, 2018) y GLPK. En GLPK se fijó un límite de tiempo de 300 segundos (5 minutos). Los resultados se resumen en la Tabla 2.

Tabla 2: Comparativa de resolución del MCP sobre keller4: HiGHS vs GLPK.

Métrica	HiGHS	GLPK
Mejor solución encontrada (ω)	11	11
Óptimo conocido	11	11
Tiempo de resolución	12–18 s (según ejecución)	> 300 s (<i>timeout</i>)
Estado del solver	Óptimo	Factible (sin probar optimalidad)

HiGHS resolvió el modelo hasta optimalidad en un tiempo de entre 12 y 18 segundos (dependiendo de la ejecución), alcanzando el óptimo conocido $\omega = 11$. GLPK, en cambio, encontró un clique factible del mismo tamaño pero no logró cerrar la brecha de optimalidad dentro del límite de 300 segundos, terminando con estado *feasible*. Esto evidencia el impacto de las técnicas modernas de resolución MIP (planos de corte, heurísticas, preprocesamiento) implementadas en HiGHS frente a la base más austera de GLPK.

La solución fue validada verificando que el subgrafo inducido por los 11 vértices del clique es completo, es decir, contiene exactamente $\binom{11}{2} = 55$ aristas, todas presentes en el grafo original.

4.4. Análisis de la relajación LP

Para evaluar la calidad de la formulación, se resolvió la relajación lineal continua del modelo (reemplazando $x_i \in \{0, 1\}$ por $x_i \in [0, 1]$). Los resultados se presentan en la Tabla 3.

Tabla 3: Comparación entre solución entera y relajación LP.

Métrica	Valor
Cota LP (relajación)	85.50
Solución entera (ILP)	11
Brecha de integralidad	87.1 %

La brecha de integralidad de 87.1 % indica que la relajación LP proporciona una cota superior muy débil para esta instancia. Este fenómeno es bien conocido en la literatura: la formulación estándar del MCP basada en restricciones de pares no adyacentes produce relajaciones LP intrínsecamente débiles, especialmente en grafos con estructura regular (Bomze et al., 1999; Nemhauser & Wolsey, 1988). Esto implica que el solver debe explorar un árbol de Branch & Bound extenso para cerrar la brecha entre la cota LP y la solución entera óptima.

4.5. Visualización

La Figura 2 muestra el grafo keller4 con el clique máximo destacado en rojo. Los vértices pertenecientes al clique se identifican con mayor tamaño y las aristas entre ellos se resaltan. La distribución de grados del grafo muestra que los vértices del clique poseen grados superiores al promedio, lo cual es consistente con la estructura esperada de un clique en un grafo denso.

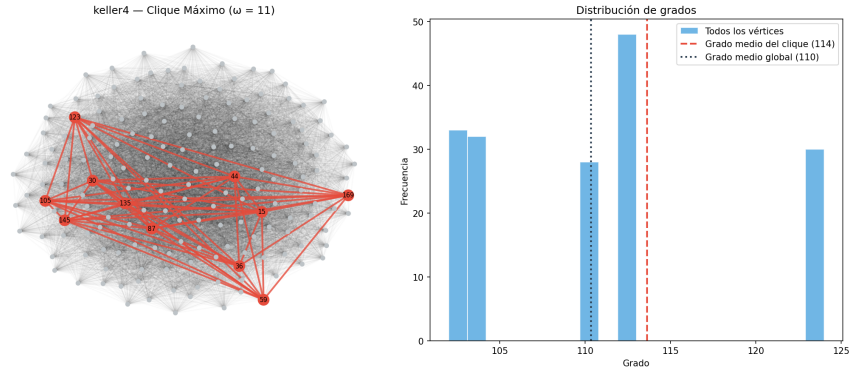


Figura 2: Grafo keller4 con el clique máximo resaltado (vértices del clique en rojo y mayor tamaño).

5. Discusión

5.1. Efectividad de la formulación ILP

La formulación ILP estándar del MCP, basada en restricciones sobre pares no adyacentes (Ecuaciones (2)–(4)), demostró ser suficiente para resolver la instancia keller4 hasta optimalidad en un tiempo computacional razonable (entre 12 y 18 s según ejecución). Esto valida el enfoque de resolución exacta mediante solvers MIP modernos para instancias de tamaño moderado (Wolsey, 1998).

Sin embargo, la elevada brecha de integralidad (87.1 %) revela una limitación estructural de esta formulación. La relajación LP no captura adecuadamente la combinatoria del problema, lo que obliga al solver a realizar un esfuerzo significativo de ramificación. En la literatura se han propuesto formulaciones reforzadas mediante desigualdades de clique, restricciones de rango y coloreo fraccionario que producen relajaciones más ajustadas (Bomze et al., 1999; Schrijver, 1986).

5.2. Rol del solver HiGHS

HiGHS (Huangfu & Hall, 2018) demostró ser significativamente más eficiente que GLPK para esta clase de problemas. Como muestra la Tabla 2, GLPK encontró un clique factible de tamaño $\omega = 11$ pero no logró probar optimalidad dentro del límite de 300 segundos, mientras que HiGHS cerró la brecha de optimalidad en un tiempo de entre 12 y 18 segundos según la ejecución. Esta diferencia se atribuye a las técnicas avanzadas implementadas en HiGHS: preprocesamiento, generación de cortes (Gomory, MIR), heurísticas de factibilidad y estrategias de ramificación adaptativas.

5.3. Ventajas del enfoque con Pyomo

El uso de Pyomo como framework de modelado algebraico ofrece ventajas metodológicas relevantes (Bynum et al., 2021):

- **Separación modelo-solver:** La formulación matemática es independiente del solver utilizado. Cambiar de HiGHS a CPLEX o Gurobi requiere modificar una

sola línea de código.

- **Legibilidad:** El modelo en Pyomo refleja directamente la formulación matemática, facilitando la verificación y reproducibilidad.
- **Extensibilidad:** Es directo agregar restricciones adicionales (simetría, cortes de usuario) sin reestructurar el código.

5.4. Comparación con enfoques alternativos

Se consideraron dos enfoques complementarios durante el desarrollo:

1. **Branch & Bound manual con PyBnB** (Hackebeit, 2020): Implementación explícita del algoritmo B&B usando `scipy` (Virtanen et al., 2020) para resolver las relajaciones LP en cada nodo. Este enfoque ofrece control total sobre la estrategia de ramificación y poda, pero requiere mayor complejidad de implementación.
2. **Pyomo + solver MIP:** El solver gestiona internamente todo el proceso de B&B. Se pierde visibilidad sobre el árbol de búsqueda, pero se gana eficiencia y robustez.

Para los objetivos del presente trabajo —demostrar la aplicación de optimización exacta al MCP—, el enfoque con Pyomo resulta más adecuado por su claridad y concisión, delegando la complejidad algorítmica al solver.

5.5. Limitaciones

Las principales limitaciones identificadas son:

- La formulación ILP estándar no escala bien a instancias de gran tamaño ($|V| > 500$) debido al número cuadrático de restricciones ($O(|\bar{E}|)$).
- La brecha de integralidad elevada sugiere que formulaciones alternativas o técnicas de corte serían necesarias para instancias más difíciles.
- El análisis se limita a una instancia del benchmark DIMACS; un estudio más completo requeriría evaluar múltiples familias de grafos.

6. Conclusiones

A partir del trabajo realizado, se establecen las siguientes conclusiones:

1. **Formulación exitosa:** El Problema del Clique Máximo fue formulado correctamente como un Programa Lineal Entero Binario y resuelto hasta optimalidad mediante el solver HiGHS, obteniendo $\omega = 11$ para la instancia keller4, coincidiendo con el óptimo conocido del benchmark DIMACS.
2. **Validez del enfoque de modelado algebraico:** Pyomo demostró ser una herramienta efectiva para la implementación de modelos de optimización combinatoria, permitiendo una traducción directa de la formulación matemática a código ejecutable sin sacrificar legibilidad ni generalidad.
3. **Importancia del solver:** La elección del solver tiene un impacto determinante en la viabilidad práctica. HiGHS probó optimalidad en un tiempo de entre 12 y 18 segundos según la ejecución, mientras que GLPK encontró un clique factible del mismo tamaño pero no logró cerrar la brecha en más de 5 minutos. Esto evidencia que las técnicas modernas de resolución MIP (cortes, heurísticas, preprocesamiento) son esenciales para abordar problemas NP-duros de tamaño real.
4. **Debilidad de la relajación LP:** La brecha de integralidad de 87.1% confirma que la formulación estándar basada en restricciones de pares produce relajaciones LP débiles para el MCP. Esto es consistente con los resultados teóricos de la literatura (Bomze et al., 1999) y motiva el uso de desigualdades válidas adicionales en trabajos futuros.
5. **Reproducibilidad:** La implementación completa en un Jupyter Notebook (Kluyver et al., 2016) con dependencias open-source (Python (Python Software Foundation, 2024), Pyomo, HiGHS, NetworkX (Hagberg et al., 2008)) garantiza la reproducibilidad de los resultados.

6.1. Trabajo futuro

Se identifican las siguientes líneas de extensión:

- Evaluar formulaciones reforzadas con desigualdades de clique y cortes de odd-hole.
- Comparar el rendimiento sobre múltiples familias del benchmark DIMACS (C, brock, p_hat, gen).
- Implementar un enfoque híbrido que combine heurísticas (greedy, búsqueda local) con la verificación exacta de optimalidad.
- Explorar la formulación cuadrática de Motzkin-Straus (Motzkin & Straus, 1965) como alternativa de resolución.

7. Apéndice

7.1. Entorno de ejecución

La Tabla 4 detalla las herramientas y versiones utilizadas en la implementación experimental.

Tabla 4: Herramientas de software utilizadas.

Herramienta	Versión	Propósito
Python (Python Software Foundation, 2024)	3.12+	Lenguaje de programación
Pyomo (Bynum et al., 2021)	6.7+	Modelado algebraico de optimización
HiGHS (Huangfu & Hall, 2018)	1.7+	Solver LP/MIP
NetworkX (Hagberg et al., 2008)	3.2+	Manipulación de grafos
NumPy (Harris et al., 2020)	1.26+	Cómputo numérico
Matplotlib (Hunter, 2007)	3.8+	Visualización
Jupyter (Kluyver et al., 2016)	7.0+	Notebook interactivo

7.2. Fragmento de código: modelo ILP en Pyomo

A continuación se presenta el código principal de la formulación del MCP en Pyomo:

```
import pyomo.environ as pyo

def build_mcp_model(G):
    model = pyo.ConcreteModel("MaximumClique")
    model.V = pyo.Set(initialize=list(G.nodes()))
    model.x = pyo.Var(model.V, domain=pyo.Binary)
    model.obj = pyo.Objective(
        expr=sum(model.x[i] for i in model.V),
        sense=pyo.maximize
    )
    model.nonadj = pyo.ConstraintList()
```



```

nodes = list(G.nodes())
for idx, i in enumerate(nodes):
    for j in nodes[idx+1:]:
        if not G.has_edge(i, j):
            model.nonadj.add(model.x[i] + model.x[j] <= 1)
return model

model = build_mcp_model(G)
solver = pyo.SolverFactory('appsi_highs')
result = solver.solve(model)

```

7.3. Instancia keller4: formato DIMACS

La instancia se distribuye en formato ASCII estándar DIMACS con la siguiente estructura:

```

c keller4.clq
c Keller graph on 4 dimensions
p col 171 9435
e 1 2
e 1 3
...

```

Donde `p col 171 9435` define un grafo de 171 vértices y 9435 aristas, y cada línea `e u v` define una arista entre los vértices u y v . El archivo completo está disponible en el repositorio del proyecto.

Referencias

- Bomze, I. M., Budinich, M., Pardalos, P. M., & Pelillo, M. (1999). The maximum clique problem. *Handbook of Combinatorial Optimization*, 4, 1-74.
- Bynum, M. L., Hackebeil, G. A., Hart, W. E., Laird, C. D., Nicholson, B. L., Siirola, J. D., Watson, J.-P., & Woodruff, D. L. (2021). Pyomo—optimization modeling in Python (3.^a ed.). *Springer Optimization and Its Applications*, 67. <https://doi.org/10.1007/978-3-030-68928-5>
- Carraghan, R., & Pardalos, P. M. (1990). An exact algorithm for the maximum clique problem. *Operations Research Letters*, 9(6), 375-382.
- Gardiner, E. J., Artymiuk, P. J., & Willett, P. (1997). Clique-detection algorithms for matching three-dimensional molecular structures. *Journal of Molecular Graphics and Modelling*, 15(4), 245-253. [https://doi.org/10.1016/S1093-3263\(97\)00089-2](https://doi.org/10.1016/S1093-3263(97)00089-2)
- Hackebeil, G. A. (2020). PyBnB: A parallel branch-and-bound engine for Python [MIT License].
- Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. *Proceedings of the 7th Python in Science Conference (SciPy2008)*, 11-15.
- Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585, 357-362. <https://doi.org/10.1038/s41586-020-2649-2>
- Huangfu, Q., & Hall, J. A. J. (2018). Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1), 119-142. <https://doi.org/10.1007/s12532-017-0130-5>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90-95. <https://doi.org/10.1109/MCSE.2007.55>
- Johnson, D. S., & Trick, M. A. (1993). The Second DIMACS Implementation Challenge: NP Hard Problems: Maximum Clique, Graph Coloring, and Satisfiability.
- Kluyver, T., Ragan-Kelley, B., Pérez, F., et al. (2016). Jupyter Notebooks—a publishing format for reproducible computational workflows.

- Lagarias, J. C., & Shor, P. W. (1992). Keller's cube-tiling conjecture is false in high dimensions. *Bulletin of the American Mathematical Society*, 27(2), 279-283. <https://doi.org/10.1090/S0273-0979-1992-00318-X>
- Mascia, F. (2014). DIMACS Benchmark Set for the Maximum Clique Problem [IRIDIA, Université Libre de Bruxelles. Consultado en abril 2026].
- Motzkin, T. S., & Straus, E. G. (1965). Maxima for graphs and a new proof of a theorem of Turán. *Canadian Journal of Mathematics*, 17, 533-540. <https://doi.org/10.4153/CJM-1965-053-6>
- Nemhauser, G. L., & Wolsey, L. A. (1988). *Integer and Combinatorial Optimization*. Wiley, New York.
- Pardalos, P. M., & Rodgers, G. P. (1992). A branch and bound algorithm for the maximum clique problem. *Computers and Operations Research*, 19(5), 363-375.
- Pardalos, P. M., & Xue, J. (1994). The maximum clique problem. *Journal of Global Optimization*, 4(3), 301-328. <https://doi.org/10.1007/bf01098364>
- Python Software Foundation. (2024). Python Language Reference, version 3.
- Schrijver, A. (1986). *Theory of Linear and Integer Programming*. John Wiley & Sons, Chichester.
- Singh, K. K., & Govinda, L. (2014). A Simple and Efficient Heuristic Algorithm For Maximum Clique Problem. *2014 IEEE 8th International Conference on Intelligent Systems and Control (ISCO)*, 269-273. <https://doi.org/10.1109/ISCO.2014.7103958>
- Szabó, S., & Záválnij, B. (2018). A Different Approach to Maximum Clique Search. *2018 20th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC)*, 310-316. <https://doi.org/10.1109/SYNASC.2018.00055>
- Virtanen, P., Gommers, R., Oliphant, T. E., et al. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261-272. <https://doi.org/10.1038/s41592-019-0686-2>
- Wolsey, L. A. (1998). *Integer Programming*. John Wiley & Sons, New York.